

Simulator paralel distribuit pentru Conway's Game of Life folosind MPI

Masterand: Ilculesei-Meglei Ștefan

Grupa:3711a

Introducere în automate celulare și Conway's Game of Life

Conway's Game of Life este unul dintre cele mai cunoscute automate celulare bidimensionale. Deși regulile sale sunt extrem de simple, dinamica rezultată poate produce comportamente complexe, pattern-uri stabile, oscilatoare și structuri mobile. Fiecare celulă din grilă se afla într-una dintre două stări posibile, vie sau moartă, iar tranziția dintre generații depinde exclusiv de numărul de vecini vii din jurul său. Aceasta combinație dintre reguli locale simple și comportament global emergent face problema atractivă atât din punct de vedere didactic, cât și din punct de vedere al experimentării algoritmice.

Din perspectiva programării paralele, Game of Life reprezintă un studiu de caz foarte potrivit. Fiecare generație necesită actualizarea tuturor celulelor, iar noua stare a unei celule depinde doar de o vecinătate mică, de tip stencil, formată din opt vecini. Prin urmare, problema poate fi descompusă natural în subdomenii distribuite între mai multe procese MPI, fiecare proces calculând local o porțiune a grilei și comunicând doar informațiile de frontieră către procesele vecine. Apar astfel toate conceptele esențiale din programarea distribuită: descompunere de domeniu, schimb halo, cost de comunicație, sincronizare, echilibrare a încărcării și analiza scalabilității.

Tema proiectului a fost implementarea unui simulator distribuit pentru Conway's Game of Life pe grile 2D toroidale de dimensiuni mari, folosind limbajul C și biblioteca MPI. Condiția de tor periodic este importantă deoarece elimină frontierele fixe și face ca vecinătatea să fie tratată uniform pe toată suprafața grilei: prima linie comunică cu ultima, iar prima coloană comunică cu ultima. Aceasta alegere simplifică definirea matematică a problemei, dar introduce cerințe clare pentru implementarea paralelă, mai ales în varianta distribuită.

În cadrul proiectului au fost dezvoltate două direcții principale. Prima direcție este o versiune serială de referință, utilizată atât pentru validare funcțională, cât și pentru comparația de performanță. A doua direcție este versiunea paralelă MPI, care include două strategii de descompunere: o descompunere 1D pe linii și o descompunere 2D pe blocuri. Comunicația dintre procese este implementată în mod non-blocking, cu `'MPI_Isend'`, `'MPI_Irecv'` și `'MPI_Wait'`, pentru a permite suprapunerea parțială dintre calcul și schimbul de halo.

Pe lângă componenta de calcul, proiectul include și facilități practice importante: inițializări deterministe pentru pattern-uri cunoscute, validare serial vs MPI, export CSV pentru benchmark-uri, salvare de snapshot-uri PGM și o interfață grafică opțională pentru demo. Totuși, accentul principal al proiectului rămâne pe corectitudine, proiectare paralelă și analiza de performanță.

Algoritmul serial

Versiunea serială reprezintă baza funcțională a întregului proiect. Ea definește regulile Game of Life, modelul de date pentru grilă și modul de evoluție între generații. Implementarea folosește două buffere 2D conceptuale, reprezentate în memorie liniar, unul pentru starea curentă și unul pentru starea următoare. La fiecare pas, toate celulele sunt parcurse, se calculează numărul de vecini vii și se aplică regulile standard Conway:

- o celulă vie cu mai puțin de doi vecini moare, prin subpopulare
- o celulă vie cu doi sau trei vecini vii supraviețuiește
- o celulă vie cu mai mult de trei vecini vii moare, prin suprapopulare

- o celulă moartă cu exact trei vecini vii devine vie

Pentru a implementa frontierele periodice, accesarea vecinilor se face modular, astfel încât indicii care ies din interval sunt readuși în domeniul valid. Aceasta înseamnă că fiecare celulă are întotdeauna opt vecini definiți, indiferent de poziția sa în grilă.

Pseudocodul de baza al versiunii seriale este următorul:

```

initializare grilă curentă
initializează grilă următoare
pentru fiecare generație:
    pentru fiecare celulă:
        numără cei 8 vecini vii
        aplică regulile Conway
        scrie rezultatul în grila următoare
    interschimbă grila curentă cu grila următoare

```

Figura 1. Pseudocod soluție serială

Complexitatea temporală a algoritmului serial este proporțională cu numărul total de actualizări efectuate, adică:

$$T_{\{serial\}} = O(steps \cdot width \cdot height)$$

Complexitatea spațială este dominată de cele două buffere ale grilei:

$$S_{\{serial\}} = O(width \cdot height)$$

Aceasta implementare este suficientă pentru grile mici și medii, dar devine rapid insuficientă pentru dimensiuni foarte mari sau pentru un număr mare de generații. Timpul total de execuție crește liniar cu volumul problemei, iar memoria necesară pentru grile mari poate deveni restrictivă pe un singur proces. Din acest motiv, versiunea serială este folosită în principal ca referință de corectitudine și ca baza pentru calculul speedup-ului în analiza paralelă.

Un avantaj important al implementării seriale este determinismul. Pentru aceeași dimensiune a grilei, același seed și același pattern inițial, rezultatul este reproductibil bit cu bit. Acest aspect a fost exploatat direct în proiect pentru validarea variantei MPI. În plus, logica de simulare a fost extrasă într-un backend comun de tip engine, astfel încât interfață serială și interfață grafică opțională să folosească exact aceeași regulă de evoluție, fără duplicare de cod.

Modele de paralelizare și provocări MPI

Paralelizarea Game of Life în regim de memorie distribuită presupune impartirea grilei între mai multe procese MPI. Fiecare proces este responsabil pentru un subdomeniu local și calculează independent noile stări pentru celulele din acel subdomeniu. Totuși, deoarece actualizarea unei celule depinde de vecini, procesele trebuie să facă schimb de informații pentru zonele de frontieră, la fiecare generație. Acest schimb poartă numele de halo exchange sau ghost cell exchange.

În proiect au fost implementate două strategii de descompunere.

Prima strategie este descompunerea 1D pe linii. Grilă globala este împărțită pe benzi orizontale, fiecare proces primind un număr aproximativ egal de linii consecutive. Avantajul principal al acestei metode este simplitatea. Fiecare proces trebuie să comunice doar cu vecinul de sus și cu vecinul de jos pentru a primi liniile halo necesare. Dezavantajul este că suprafața de comunicație nu scade foarte bine atunci când numărul de procese crește, mai ales dacă subdomeniile devin foarte subțiri.

A doua strategie este descompunerea 2D pe blocuri. Grilă globala este împărțită în subblocuri, iar fiecare proces are vecini pe direcțiile nord, sud, est și vest, plus schimb de colțuri pentru a menține vecinătatea completă. Aceasta strategie este mai apropiată de un stencil 2D natural și poate reduce raportul suprafață/volum al subdomeniului față de descompunerea 1D. În schimb, implementarea este mai complexă, deoarece presupune gestionarea mai multor direcții de comunicație și tratarea corectă a colțurilor.

Comunicația a fost implementată folosind operații non-blocking. Ideea este că procesul să posteze mai întâi recepțiile și trimiterile pentru halo, apoi să calculeze zona interioară a subdomeniului local, care nu depinde de datele noi de la vecini. Doar după aceea procesul așteaptă finalizarea comunicațiilor și calculează frontierele. Aceasta abordare reduce timpul pierdut în așteptare și urmărește să suprapună o parte din calcul cu traficul de date.

Principalele provocări ale implementării MPI au fost următoarele:

- alegerea unei partiții echilibrate, astfel încât toate procesele să aibă un volum de calcul comparabil
- tratarea frontierelor periodice, inclusiv între procesele extreme
- evitarea deadlock-urilor prin ordonarea corectă a operațiilor de comunicație
- menținerea determinismului față de varianta serială
- separarea costului de calcul de costul de comunicație pentru analiza ulterioară

Din punct de vedere conceptual, Game of Life este un exemplu clasic de problemă în care paralelizarea nu este limitată de logica regulilor locale, ci de eficiență cu care sunt gestionate dependențele dintre subdomenii. Pe măsură ce numărul de procese crește, dimensiunea subdomeniului local scade, iar ponderea relativă a comunicației crește. Tocmai acest raport dintre calcul și comunicație reprezintă unul dintre subiectele centrale ale evaluării experimentale.

Design-ul soluției paralele propuse

Implementarea finală a proiectului este organizată în jurul unui backend comun pentru logica serială și al unei intrări separate pentru execuția MPI. Fișierele sursă sunt structurate astfel încât regulile de simulare, parsarea argumentelor, inițializarea pattern-urilor și interfețele comune să fie partajate, în timp ce fluxul de execuție distribuit este concentrat în executabilul MPI.

Pentru fiecare proces MPI, subdomeniul local este stocat în două buffere: unul pentru starea curentă și unul pentru starea următoare. În plus, sunt folosite zone halo sau buffere auxiliare pentru datele primite de la vecini. La fiecare generație, fluxul de execuție este următorul:

1. se posteaza operațiile MPI_Irecv pentru datele halo
2. se posteaza operațiile MPI_Isend către vecini
3. se calculează zona interioara a subdomeniului local
4. se așteaptă finalizarea comunicărilor cu MPI_Wait sau MPI_Waitall
5. se calculează celulele de frontieră folosind halo-ul actualizat
6. se interschimba bufferul curent cu bufferul urmator

Figura 2. Fluxul de execuție a soluției paralele

În descompunerea 1D, fiecare proces comunica doar liniile extreme ale subdomeniului sau. În descompunerea 2D, datele schimbate includ linii, coloane și colturi, astfel încât fiecare proces să poată reconstrui local vecinătatea necesară pentru frontieră să completa. Pentru varianta 2D au fost introduse și mecanisme de tip cartezian sau echivalente, pentru a administra mai ușor vecinii logici ai fiecărui proces.

Un aspect important de proiectare a fost păstrarea unei referințe seriale robuste. Executabilul MPI poate rula cu opțiunea de validare, caz în care rezultatul distribuit este comparat pe rank-ul 0 cu rezultatul obținut de backend-ul serial pentru aceeași configurație. Aceasta comparație a fost esențială în fază de dezvoltare, în special pentru verificarea corectă a frontierelor toroidale și a schimburilor halo.

Pe langa componenta de simulare, proiectul include facilități auxiliare utile pentru experimentare și documentare. Exista export CSV pentru automatizarea benchmark-urilor, export PGM pentru salvarea stării finale sau a snapshot-urilor intermediare și o componenta grafica SDL2 care folosește același backend serial pentru demo vizual. Din punctul de vedere al temei, aceste facilități nu înlocuiesc analiza MPI, dar cresc reproductibilitatea și ușurința prezentării.

Din perspectiva ingineriei software, o decizie benefică a fost introducerea unui API de tip engine pentru backend-ul serial. Aceasta abstracție permite reutilizarea logicii de simulare în mai multe front-end-uri și reduce riscul apariției unor comportamente diferite între executabile. În plus, un subsistem centralizat de logging face mai ușor debugging-ul în execuțiile MPI cu mai multe procese.

Analiza teoretică a performanței

Performanța unei implementări paralele pentru Game of Life poate fi descrisă ca o combinație între costul local de calcul și costul global al comunicației. Pentru un număr de procese p , timpul total poate fi aproximat prin relația:

$$T_p \approx T_{calc}(n, p) + T_{comm}(n, p) + T_{sync}(p),$$

unde T_{calc} reprezintă timpul necesar actualizării celulelor locale, T_{comm} reprezintă timpul pentru schimbul de halo, iar T_{sync} acoperă costurile de așteptare și sincronizare dintre procese.

Pentru o grilă fixă, cazul de strong scaling urmărește cum scade timpul total atunci când crește numărul de procese. În mod ideal, dacă partea serială ar fi neglijabilă și comunicația nu ar costa nimic, s-ar obține:

$$T_p \approx \frac{T_1}{p}$$

și deci speed-ul ideal ar fi:

$$S_p = \frac{T_1}{T_p} \approx p$$

Eficiență paralelă este definită prin:

$$E_p = \frac{S_p}{p}$$

În practică, speedup-ul este limitat de comunicație, de sincronizare și de faptul că dimensiunea subdomeniului local scade pe măsură ce crește p . În momentul în care subdomeniile devin prea mici, raportul dintre suprafața care necesită schimb halo și volumul de calcul local devine nefavorabil.

Pentru weak scaling, problema este redimensionată astfel încât volumul de lucru local pe proces să rămână aproximativ constant. În acest caz, interesul principal este dacă timpul total rămâne stabil pe măsură ce crește numărul de procese. O abatere mică față de timpul de baza indică un comportament bun de scalare, în timp ce o creștere puternică sugerează că traficul de comunicație și sincronizarea devin dominante.

Un indicator foarte util în acest proiect este procentul de timp petrecut în comunicație:

$$P_{comm} = \frac{T_{comm}}{T_{total}} \cdot 100$$

Acest indicator permite interpretarea directă a scalabilității. Dacă P_{comm} rămâne redus, implementarea folosește bine resursele și câștigă din paralelizare. Dacă P_{comm} crește rapid, atunci eficiența se degradează, chiar dacă speedup-ul absolut poate continua să crească într-o anumită măsură.

Din punct de vedere calitativ, descompunerea 2D ar trebui să devină mai avantajoasă decât cea 1D atunci când dimensiunea grilei și numărul de procese cresc suficient, deoarece reduce suprafața de frontieră raportată la volumul subdomeniului. Totuși, pentru numere mici sau moderate de procese, această superioritate nu este garantată automat, deoarece apare și un cost suplimentar de administrare a mai multor direcții de comunicație. De aceea, analiza experimentală trebuie să compare direct cele două variante.

Rezultate experimentale

Platforma experimentală și metodologia de test

Experimentele au fost realizate pe Linux, folosind compilare C standard și execuție MPI prin `mpirun`. Timpul total, timpul de comunicație și timpul de calcul au fost măsurate în cod cu `MPI\_Wtime`, iar rezultatele au fost exportate în format CSV pentru procesare ulterioară. Pentru generarea seturilor de date au fost rulate teste de strong scaling și weak scaling pentru 1, 2, 4 și 8 procese MPI.

Configurația folosită pentru benchmark-urile din această versiune a raportului a fost:

- pattern inițial random
- seed 7, pentru reproductibilitate
- densitate inițială 0.30
- 200 generații pentru fiecare rulare;
- grilă fixă 1024 x 1024 pentru strong scaling
- grilă 1024 x (1024 * p) pentru weak scaling

În mediul local de test, rulările cu 8 procese au necesitat opțiunea `--oversubscribe`, deoarece numărul de sloturi MPI disponibile implicit era mai mic decât numărul de procese solicitate. Acest aspect ține de mediul de execuție și nu de corectitudinea aplicației.

Validarea corectitudinii

Corectitudinea implementării paralele a fost verificată în mai multe etape. Mai întâi, proiectul include un script rapid de comparație pentru cazuri mici, care confirmă concordanța dintre serial și MPI. În plus, au fost rulate verificări explicite pentru ambele descompuneri, 1D și 2D, folosind opțiunea `--validate`. În toate aceste cazuri, rezultatul a fost `validation: OK`.

Acest rezultat indică faptul că pentru aceeași stare inițială, același număr de pași și aceleași reguli toroidale, varianta distribuită reproduce corect rezultatul obținut de referință serială. Validarea este importantă nu doar pentru regulile Conway, ci mai ales pentru partea sensibilă a implementării: schimbul halo și reconstrucția corectă a vecinătăților la frontieră dintre procese.

Prin urmare, interpretarea rezultatelor de performanță din secțiunile următoare se bazează pe implementări deja verificate funcțional, ceea ce permite discutarea scalabilității fără ambiguități legate de corectitudine.

Strong scaling

Pentru strong scaling, dimensiunea problemei a fost menținută fixă la `1024 x 1024`, iar numărul de procese a crescut de la 1 la 8. Rezultatele pentru descompunerea 1D sunt prezentate în Tabelul 1.

Tabelul 1. Rezultate experimentale pentru descompunerea 1D strong scaling

Procese	Timp total (s)	Speedup	Eficiență	Comunicație (%)
1	0.726170	1.000000	1.0000	0.237173
2	0.376985	1.926259	0.963129	5.024766
4	0.191502	3.791968	0.947992	5.402399
8	0.148528	4.889107	0.611138	37.872396

Rezultatele arata că descompunerea 1D scalează foarte bine pana la 4 procese. Speedup-ul de aproximativ `3.79` la 4 procese și eficiență de aproximativ `0.95` indica o utilizare buna a resurselor. Situația se schimba la 8 procese, unde timpul total continua să scadă, dar eficiență se reduce semnificativ la aproximativ `0.61`. Cauza principala este creșterea brusca a ponderii comunicației, care ajunge la aproape `38%` din timpul total.

Rezultatele pentru descompunerea 2D sunt prezentate în Tabelul 2.

Tabelul 2. Rezultate experimentale pentru descompunerea 2D strong scaling

Procese	Timp total (s)	Speedup	Eficiență	Comunicație (%)
1	0.752928	1.000000	1.0000	0.572199
2	0.398657	1.888658	0.944329	5.120867
4	0.192369	3.913986	0.978496	8.260252
8	0.149530	5.035291	0.629411	36.907491

Și în cazul descompunerii 2D se observa un comportament foarte bun pana la 4 procese. La 4 procese, speedup-ul este chiar ușor mai bun decât în 1D, aproximativ `3.91`, iar eficiență se menține foarte ridicata, aproape `0.98`. La 8 procese apare același fenomen de degradare a eficienței, deși varianta 2D rămâne marginal mai buna decât 1D atât la speedup, cât și la procentul de comunicație.

Comparând direct cele doua tabele, se poate concluziona că pentru aceasta dimensiune a problemei ambele strategii sunt eficiente pana la 4 procese, iar 2D are un mic avantaj la 4 și 8 procese. Totuși, diferența nu este spectaculoasa în acest interval, ceea ce sugerează că beneficiile descompunerii 2D devin mai evidente doar atunci când numărul de procese sau dimensiunea grilei cresc și mai mult.

Weak scaling

Pentru weak scaling s-a menținut fixa lățimea la 1024, iar înălțimea a fost scalată proporțional cu numărul de procese. Astfel, fiecare proces a primit aproximativ același volum de lucru local, iar obiectivul a fost observarea stabilității timpului total.

Rezultatele pentru descompunerea 1D sunt prezentate în Tabelul 3.

Tabelul 3. Rezultate experimentale pentru descompunerea 1D weak scaling

Procese	Dimensiune grilă	Timp total (s)	Timp relativ	Comunicație (%)
1	1024 x 1024	0.730974	1.000000	0.245772
2	1024 x 2048	0.739482	1.011639	1.690097
4	1024 x 4096	0.961249	1.315024	21.771396
8	1024 x 8192	1.344355	1.839127	27.114556

Pana la 2 procese, weak scaling-ul 1D este foarte bun: timpul total crește foarte puțin fata de cazul de baza. La 4 și 8 procese, creșterea timpului devine însă vizibila. Aceasta tendință este însoțita de creșterea ponderii comunicației, care depășește `21%` la 4 procese și `27%` la 8 procese. Concluzia este că descompunerea 1D își pierde treptat eficiență atunci când creșterea dimensiunii globale duce la un trafic halo mai semnificativ.

Rezultatele pentru descompunerea 2D sunt prezentate în Tabelul 4.

Tabelul 4. Rezultate experimentale pentru descompunerea 2D weak scaling

Procese	Dimensiune grilă	Timp total (s)	Timp relativ	Comunicație (%)
1	1024 x 1024	0.774534	1.000000	0.596449
2	1024 x 2048	0.860951	1.111572	11.333042
4	1024 x 4096	0.818079	1.056220	11.102717
8	1024 x 8192	1.258890	1.625350	35.186159

Aceste rezultate sunt interesante deoarece la 4 procese varianta 2D are un timp relativ mai bun decât varianta 1D, în ciuda unui cost de comunicație deja vizibil. Acest lucru sugerează că împărțirea bidimensională folosește mai eficient geometria problemei în regimul intermediar. Totuși, la 8 procese comunicația devine și aici foarte costisitoare, depășind 35% din timpul total, iar timpul relativ ajunge la aproximativ 1.63.

Per ansamblu, weak scaling-ul confirmă concluzia observată și în strong scaling: descompunerea 2D poate oferi un comportament puțin mai bun într-un regim mediu de paralelism, dar pentru un număr suficient de mare de procese costul comunicației începe să domine și în aceasta varianta.

Calcul vs comunicație

Una dintre cele mai utile observații experimentale rezulta din separarea explicită dintre `total\_seconds`, `communication\_seconds` și `computation\_seconds`. Aceasta separare permite cuantificarea clară a impactului schimbului halo asupra performanței globale.

În ambele variante, la 1 proces procentul de comunicație este foarte mic, practic neglijabil, deoarece nu există trafic real între procese. La 2 și 4 procese, costul de comunicație crește, dar rămâne suficient de redus pentru ca speedup-ul să fie în continuare foarte bun. În acest interval, implementarea beneficiază de faptul că fiecare proces are încă suficient calcul local pentru a amortiza transferurile de halo.

La 8 procese apare punctul în care scalabilitatea începe să se degradeze vizibil. În strong scaling, comunicația ajunge la aproximativ 37.87% în varianta 1D și 36.91% în varianta 2D. Aceste valori explică de ce eficiența scade brusc spre 0.61 - 0.63, chiar dacă timpul total absolut continuă să scadă. În weak scaling, creșterea procentului de comunicație este de asemenea clară, mai ales în varianta 2D la 8 procese, unde acesta depășește 35%.

Concluzia acestei subsecțiuni este că implementarea paralelă este bine optimizată pentru un număr mic și mediu de procese, dar intra într-un regim dominat de comunicație atunci când dimensiunea subdomeniului local devine prea mică raportat la costul halo exchange-ului. Acest rezultat este perfect coerent cu modelul teoretic discutat anterior și confirmă faptul că optimizarea unei aplicații distribuite nu înseamnă doar împărțirea muncii, ci și minimizarea overhead-ului de comunicație.

Test complementar pe grilă mare (10k x 10k)

Pe lângă seturile de benchmark pentru strong scaling și weak scaling, a fost rulat și un test complementar pe grilă mare (10000 x 10000), salvat în fisierul coverage/large_grid_10k.log și reproductibil prin scriptul scripts/run_large_grid_check.sh.

Tabelul 5 Test de scalare pe grilă foarte mare (10k x 10k)

VARIANTĂ	Configurație	Rezultat
Serial	10000 x 10000, 5 pași	total seconds = 20.625302
MPI 2D	4 rank-uri, 10000 x 10000, 5 pași	total=0.607974820, comm=0.021683234, comp=0.599690499

Grafice

Graficele pentru secțiunea de performanță au fost generate în format SVG, pe baza fișierelor CSV de sumar.

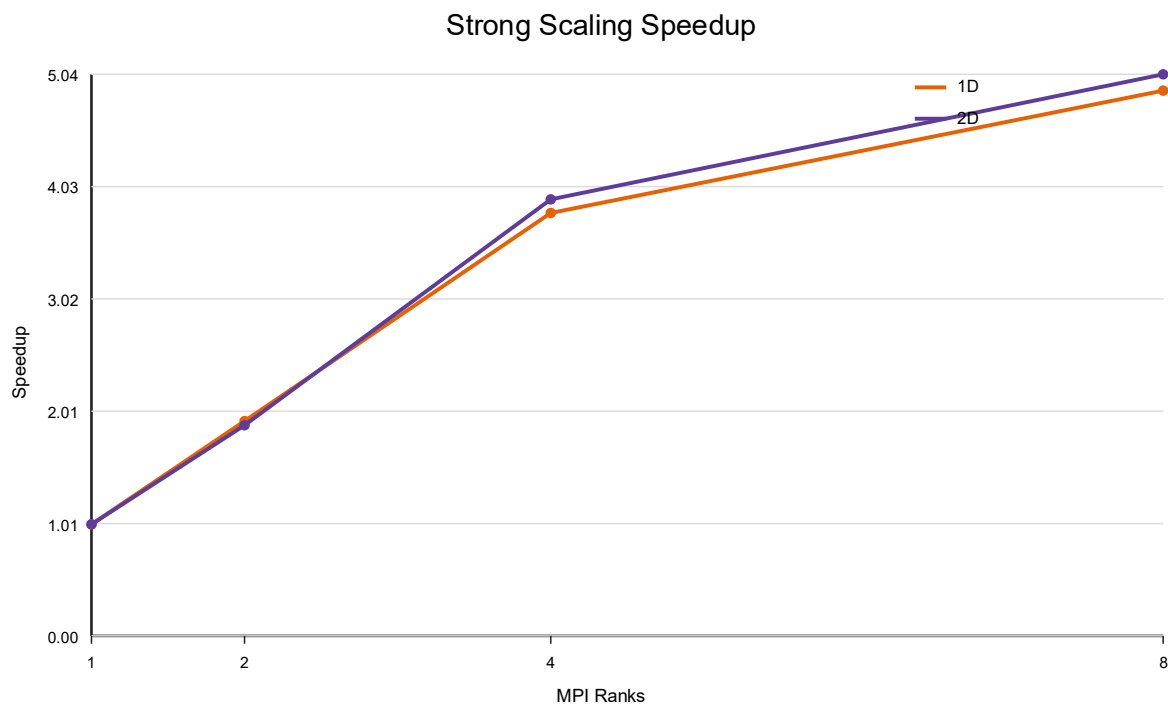


Figura 3. Strong scaling - speedup (1D vs 2D)

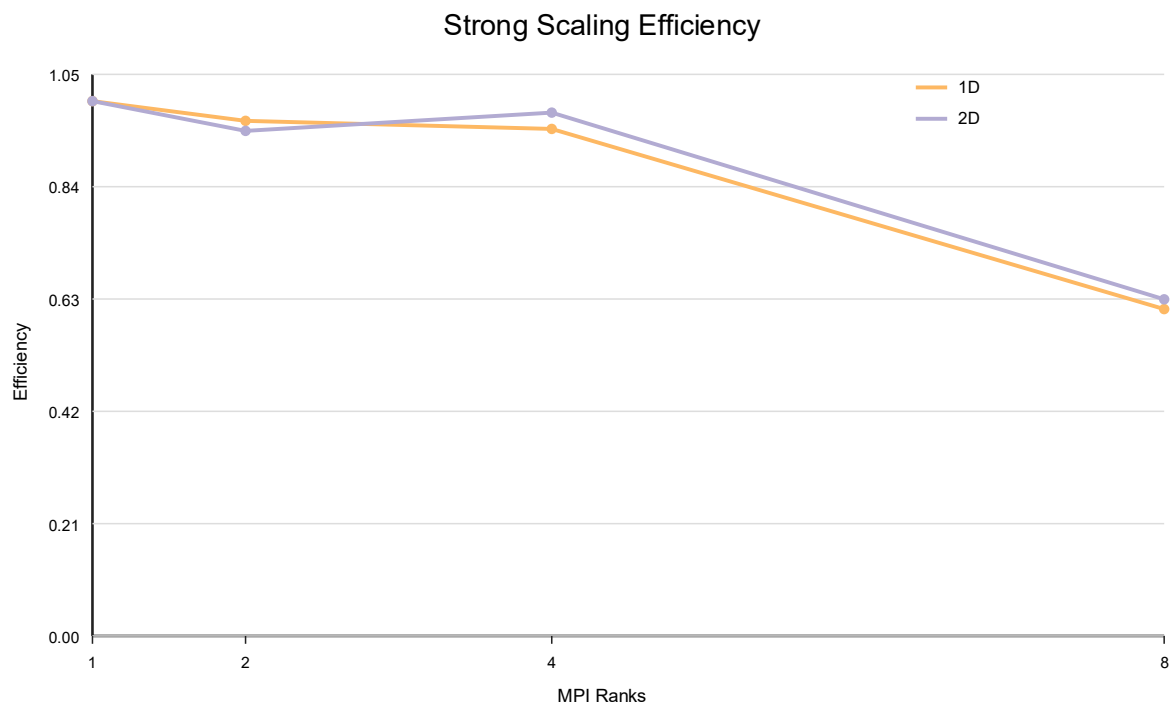


Figura 4. Strong scaling - eficiență (1D vs 2D)

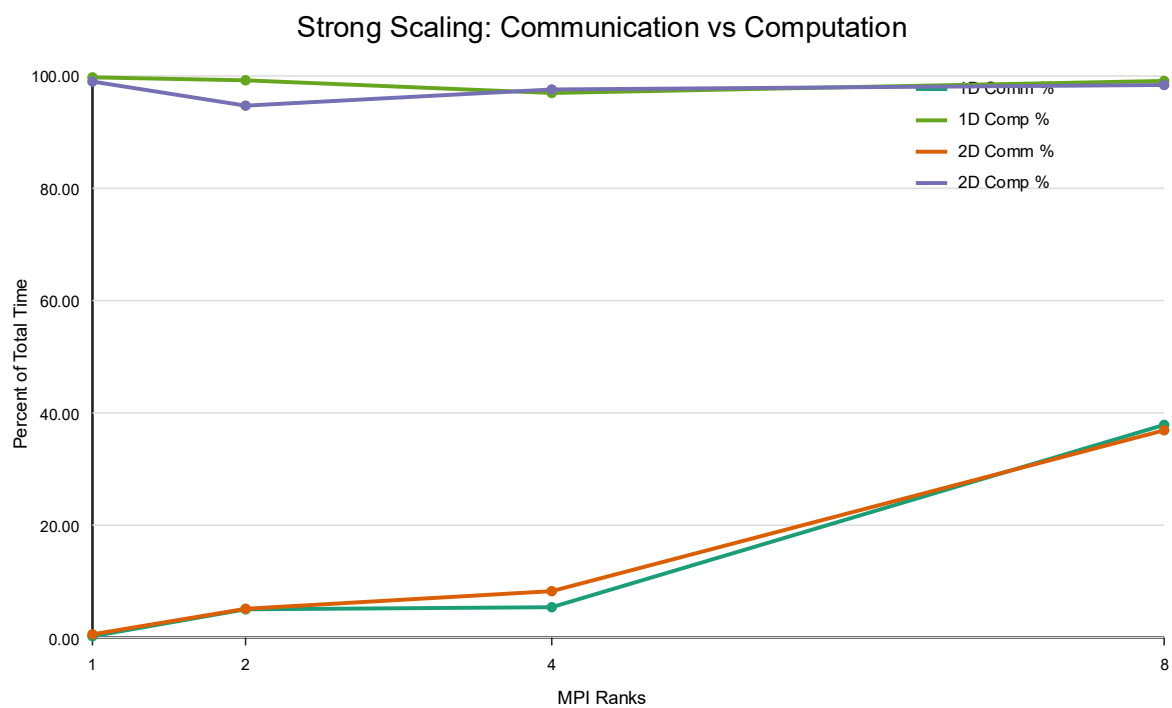


Figura 5. Strong scaling - comunicație vs calcul

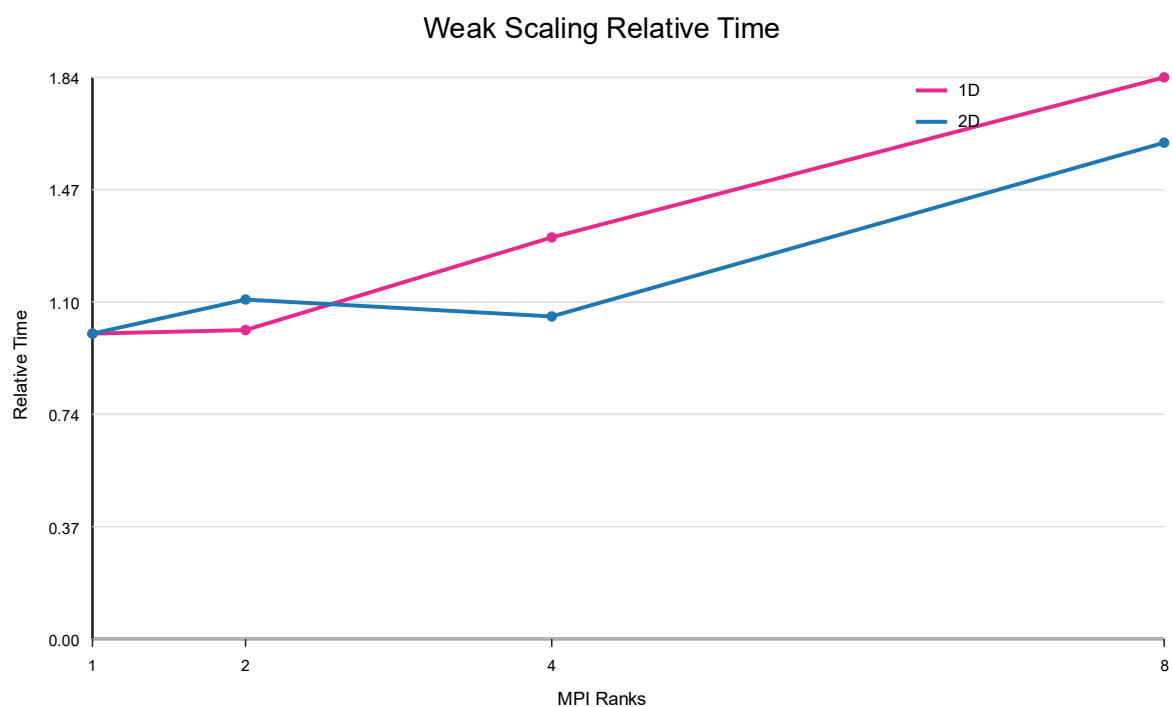


Figura 6. Weak scaling - timp relativ

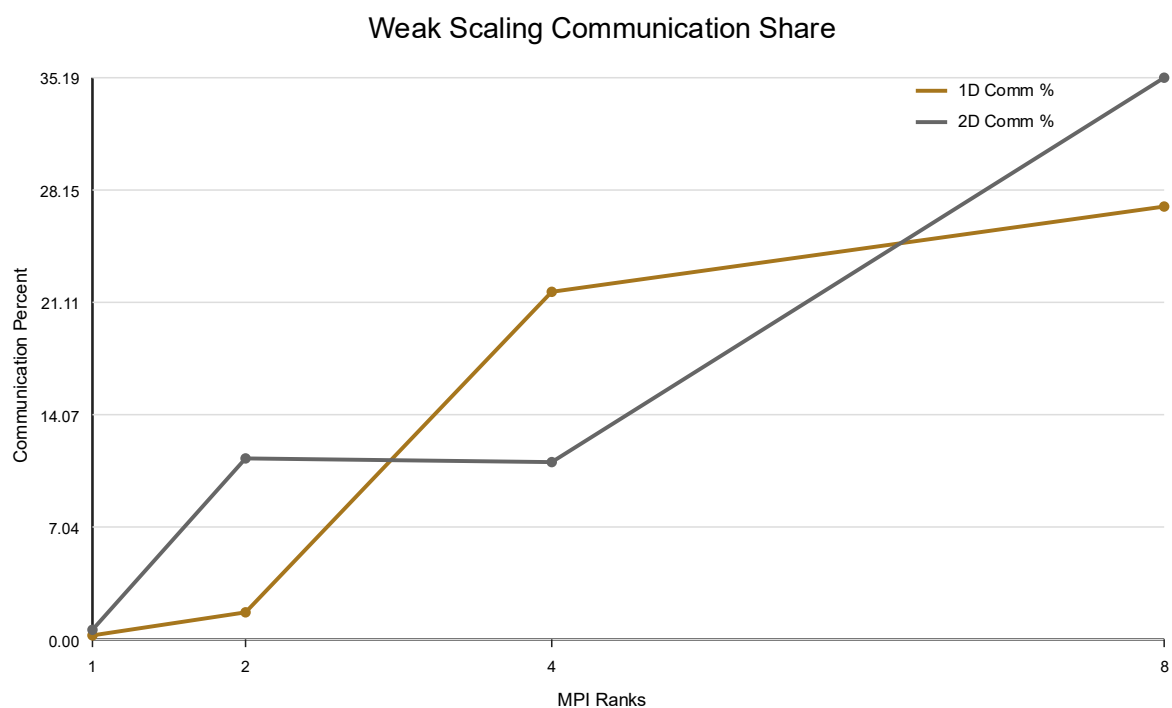


Figura 7. Weak scaling - procent comunicație

.Concluzii, limitari și posibile extensii

Proiectul a demonstrat că Conway's Game of Life este o tema foarte potrivita pentru studierea programării paralele distribuite cu MPI. Deși regulile problemei sunt simple și locale,

implementarea eficiență pe mai multe procese necesita o combinație de decizii corecte privind descompunerea de domeniu, schimbul de halo, sincronizarea și analiza performanței. Prin urmare, proiectul a permis exersarea unor concepte MPI esențiale într-un context intuitiv și ușor de verificat vizual.

Din punct de vedere funcțional, obiectivele principale au fost atinse. A fost dezvoltată o versiune serială de referință, o versiune MPI cu descompunere 1D și o versiune MPI cu descompunere 2D, toate respectând frontierele toroidale. Comunicația a fost implementată non-blocking, iar corectitudinea a fost validată prin comparație directă cu referință serială. De asemenea, proiectul include facilități practice pentru reproductibilitate și analiza, precum export CSV, snapshot-uri PGM și logging centralizat.

Din punct de vedere experimental, rezultatele arată o scalabilitate foarte bună până la 4 procese pentru ambele strategii de descompunere. Descompunerea 2D oferă în general un ușor avantaj în regimul mediu, mai ales în strong scaling la 4 procese și în weak scaling la 4 procese. Totuși, la 8 procese, în ambele variante comunicația devine o componentă majoră a timpului total, iar eficiența se degradează semnificativ. Această concluzie este firească și confirmă așteptarea teoretică potrivit căreia raportul calcul/comunicație devine nefavorabil atunci când subdomeniile locale devin prea mici.

Limitarea principală a acestei evaluări este mediul de test local. Deși rezultatele sunt relevante și suficiente pentru analiza proiectului, ele nu reproduc complet condițiile unui cluster dedicat, unde latența, topologia rețelei și distribuția reală a resurselor pot influența comportamentul la scări mai mari. În plus, benchmark-urile prezentate folosesc un singur tip de inițializare și un singur set principal de dimensiuni, ceea ce înseamnă că o evaluare și mai completă ar putea include și alte configurații.

Că direcții de continuare, proiectul poate fi extins în mai multe moduri. O primă direcție este combinarea MPI cu OpenMP pentru paralelizare hibridă intra-nod și inter-nod. O a doua direcție este accelerarea cu GPU, utilă mai ales pentru actualizarea stencil pe grile foarte mari. Alte extensii posibile includ suport pentru input din fișiere mari, generalizarea regulilor automatului celular sau extinderea către grile 3D. Din punct de vedere al interfețelor, componenta grafică opțională și un eventual terminal UI pot îmbunătăți prezentarea proiectului, deși nu modifică esența evaluării MPI.

În concluzie, proiectul oferă o implementare corectă, reproductibilă și bine instrumentată pentru Conway's Game of Life distribuit cu MPI. Rezultatele experimentale susțin faptul că abordarea este eficientă în regimuri moderate de paralelism și ilustrează clar compromisul fundamental dintre calcul și comunicație, care stă la baza multor aplicații HPC reale.

Bibliografie

1. Gardner, M. (1970). Mathematical Games: The fantastic combinations of John Conway's new solitaire game "life"
2. Balaji, P., Gropp, W., & Thakur, R. (2017). Mlife2d – A simple, parallel implementation of Conway's "Game of Life" (tutorial ATPESC). Argonne National Laboratory. Disponibil la: <https://wordpress.cels.anl.gov/atpesc/wp->

content/uploads/sites/96/2017/08/ATPESC_2017_Track-2_2_8-1_830am_Balaji-Gropp-Thakur-Hands-on_Mlife-code-desc.pdf

3. Kjolstad, F. B., & Snir, M. (2010). Ghost Cell Pattern. In Proceedings of the 2010 Workshop on Parallel Programming Patterns (ParaPloP 2010)

4. Weeden, A. (2014). Parallelization: Conway's Game of Life (modul didactic OpenMP +MPI) Disponibil la: http://gec.di.uminho.pt/Discip/MInf/cpd1415/PCP/Problemas/LIFE/Life_Enunciado.pdf